Architectural Action Plan: Programmatic Local SEO Engine (Astro + React - JavaScript Version)

This final action plan establishes a high-converting, context-aware local SEO engine tailored for deployment within your **Google Antigravity** environment. The development agent must thoroughly inspect and audit the existing code, workspace layouts, repository styles, and design system constraints before compiling changes or generating code artifacts. All implementations must strictly preserve visual consistency and design-tokens.

Phase 1: Dynamic Data & Content Collection Orchestration

Objective: Establish a completely data-driven database layer using JavaScript to fuel our programmatic engine without repetitive, templated output.

1. Unified Local Intelligence Database (src/content/config.js)

The developer agent must write an Astro Content Collections schema using standard JavaScript (defineCollection from astro:content) that structures local pages around deep semantic relevance, hyper-local neighborhoods, and a strong hedge against information commoditization.

- **Schema Node Requirements:**
    - city: String name for dynamic variable rendering (e.g., "San Jose").
    - region: Territory classification (e.g., "Northern California").
    - neighborhoods: Array of strings defining precise, prominent local neighborhood coverage boundaries (e.g., ['Willow Glen', 'Rose Garden', 'Almaden Valley', 'West San Jose', 'Silver Creek', 'Downtown San Jose']).
    - localDispatchManager: String name of the physical territory coordinator to reinforce brand trust.
    - localPricingBaseline: Object specifying starting rates, currency fields, and regional variables.
    - lat, lng: Exact float values to automate hyper-local schema injection.
    - zipCodes: Rigorous array of covered zip-code strings mapping to the target operating market.

2. High-Quality Content Auditing & Data Extraction

The agent must scan the directory for legacy files and use text extraction variables to pull real-world entity footprints.

- **Agent Constraint:** Do not generate text copy via lazy prompts or create generic filler

text that reads like average information. The code must map actual user pain points mined from historic operational briefs or internal expert interviews.

Phase 2: Programmatic Hub-and-Spoke Dynamic Routing Matrix

Objective: Flatten crawl depth and optimize link equity distribution across a JavaScript-powered silo architecture.

1. The Dynamic Core Node (src/pages/locations/[city]/[service].astro)

The agent will configure a dynamic route engine generating precise URL structures (e.g., /locations/san-jose/maid-services/) mapping to commercial buyer intent queries.

JavaScript

```
---
// src/pages/locations/[city]/[service].astro
import { getCollection } from 'astro:content';

export async function getStaticPaths() {
  const cities = await getCollection('cities');
  const services = await getCollection('services');

  return cities.flatMap((cityEntry) => {
    return services.map((serviceEntry) => ({
      params: {
        city: cityEntry.slug,
        service: serviceEntry.slug
      },
      props: { cityEntry, serviceEntry },
    }));
  });
}

const { cityEntry, serviceEntry } = Astro.props;
// Agent must look closely at existing styles and components to inject layout seamlessly
---
```

2. Bidirectional Programmatic Internal Linking

Site architecture is governed entirely by do-follow link vectors, not subdirectory naming strings.

The agent must install a component directly above the global footer to balance internal page equity.

- **The Component Blueprint:** Create a component that automatically builds a contextual related links section mapping:
    - Spoke Node $\rightarrow$ Parent Service Hub (/maid-services/).
    - Spoke Node $\rightarrow$ Parent Location Hub (/locations/san-jose/).
    - Hub Pages $\rightarrow$ Explicit index matrices of all Child Local Spoke Nodes.
- **Constraint:** This layout matrix prevents **orphan pages** and keeps crawl depth to an optimal 2-to-3 click distance from the homepage.

Phase 3: High-Converting Local UX Layout Blueprint

Objective: Optimize user interactions and increase search dwell time through micro-conversion formatting.
The developer agent must construct each dynamic layout page using **Short, Choppy Copy** layouts to eliminate the friction points of dense walls of text. The page template is divided into six core blocks:

Layout Block Architecture

Section Block	Component Type	Dynamic / Static Elements	CRO / SEO Justification
1. Primary Hero	Static Form Layout	Dynamic H1 Injector: [Service] in [City].	Passes the **Backyard Barbecue Test** via clear, high-intent formatting.
2. Local Proof Node	Flat HTML Block	Local City Text Testimonials (No Carousels).	Google devalues clone pages; flat local reviews ensure heavy **Information Gain**.
3. Neighborhood Coverage Area	Flat HTML Grid or List	Dynamic Neighborhood Ingestion Array loop over the neighborhoods array.	Signals geographic micro-relevance to crawlers and visual validation to local users.

4. Market Specifics	Contextual MD	Dynamic local manager bios, localized region notes, service boundaries.	Strong hedge against automated commoditization; answers local buyer questions.
5. Booking Engine	React Component	Local pricing charts driven by the JavaScript data layer.	Extends session duration (dwell time) via interactive functional value.
6. Semantic FAQ Engine	JSON-LD + HTML	Local question models extracted from Search Console and PAA datasets.	Hardcodes LocalBusiness schema directly in the header with geographical coordinates.

- **Friction Reduction Rule:** Form blocks must not flag non-critical tracking loops (such as "How did you hear about us?") as required fields, preventing abandonment cycles in browser recordings.

Phase 4: Automated Testing & Validation Pipeline

Objective: Continuously run real-time audits to protect workspace context and preserve design fidelity.

1. Visual Integrity & Style Constraints

Before committing changes to the main repository layout workspace, the agent must run local design token compiler checks to ensure that no layout adjustments alter global fonts, spacing metrics, padding boundaries, or brand style guides.

2. Indexation & Technical Guardrails Audit

The agent must automatically run programmatic quality assurance routines before final file builds to verify:

- **Canonical Matching:** The rel="canonical" tag must self-reference the exact matching dynamic page URL parameters.
- **Robots Exclusions:** Check that the build output respects the root robots.txt configuration and contains no rogue blocks or conflicting index tags.
- **Schema Structural Validity:** Run structural checks to ensure the dynamically compiled header blocks conform precisely to standard schema.org syntax formats.

Antigravity Agent Directive

CRITICAL AGENT INSTRUCTION: Open, read, and cross-reference all parent system components, typography styles, layout stylesheets, and global component rules within this workspace root folder. All sandbox code generated under Phase 2 and Phase 3 must preserve existing layout contexts, maintain design-token parameters, use pure JavaScript, and function directly within the local development web server pipelines without throwing frontend framework compilation exceptions. Ensure the dynamic neighborhood data array is elegantly rendered in its respective component section layout without visually disrupting the global CSS theme.